Ray异构融合底座重构数据管道：架构演进与万卡落地实践

原创 腾讯程序员 腾讯技术工程 2026年1月7日 03:10 广东 8人



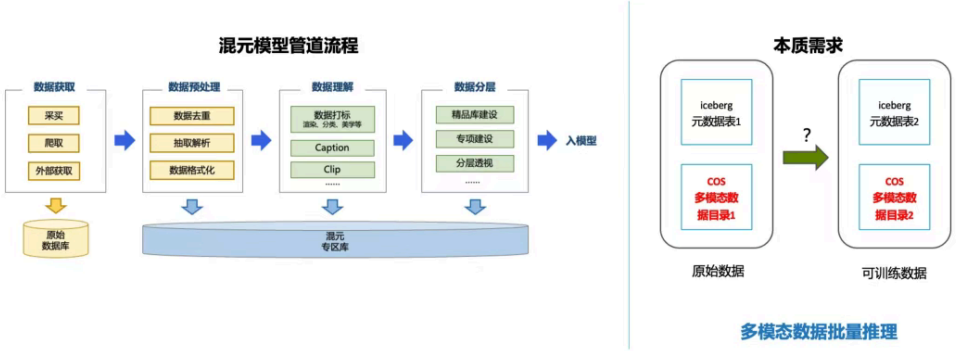
作者：gray

随着大语言模型和多模态技术的蓬勃发展，非结构化多模态数据处理已经成为数据处理的重要组成部分。面对海量数据与复杂模型带来的挑战，传统大数据引擎在异构资源调度、Python生态兼容性等方面的局限性日益凸显。本文将详述基于异构融合底座Ray重构混元数据管道的相关工作，通过云原生调度融合、计算范式统一及平台化建设，构建高效灵活的新一代AI数据底座，并深入解析我们在容错、资源利用率、规模化、可观测性等方面的优化实践。

一、传统大数据引擎在AI数据管道中的局限性

1. 数据管道的本质是离线推理

数据管道的流程如下：



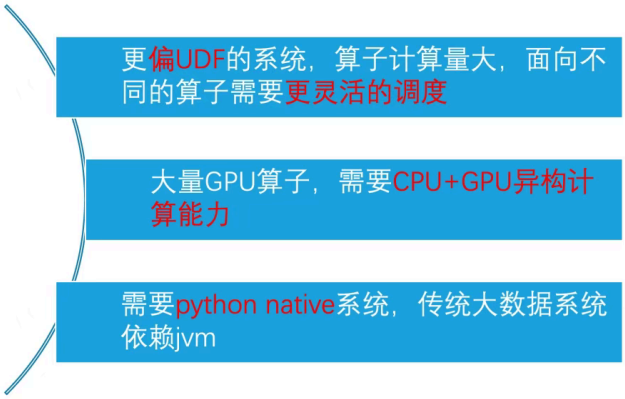
数据管道的本质需求

乍看之下，混元数据管道的形态与传统大数据ETL任务颇为相似：两者均属批处理范畴，且输入输出端通常对接湖仓表格（如Iceberg）。然而，深入剖析其内部逻辑可知，数据管道处理的主体并非表格数据本身，而是其索引的海量非结构化多模态数据（通常存储于COS等对象存储中），Iceberg表在此仅承担元数据索引的角色。

在计算负载层面，绝大部分CPU与GPU算力被消耗于基于深度学习模型的多模态数据处理环节，而非简单的元数据操作。因此，这一过程的本质应被定义为“批量推理”（Batch Inference）。相较于传统ETL，如何以更高吞吐、更高效率支撑大规模模型推理，构成了AI数据管道的本质需求与核心挑战。

2. 批量推理对计算引擎的新挑战

传统的以CPU为主的大数据引擎（如Spark、Flink）在面对AI时代的批量推理任务时，面临诸多挑战，主要体现在以下几个方面：

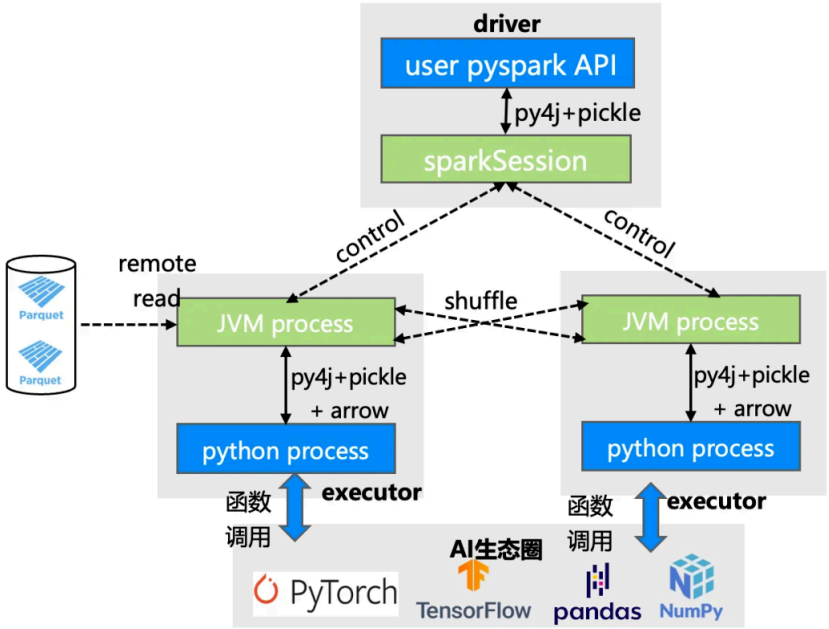


传统大数据引擎处理AI任务的挑战

- **细粒度的算子级资源调度：**模型推理算子通常计算量巨大且内存/显存敏感。传统引擎往往采用粗粒度的资源管理（如Executor级），难以满足对GPU算力和显存、CPU算力和内存进行精细化调度（UDF级）的需求。

- **CPU+GPU异构计算能力：**现代AI任务通常包含复杂的预处理（CPU密集型）、CPU模型推理（CPU密集型）、GPU模型推理（GPU密集型）。高效的数据管道需要实现CPU与GPU的流水线执行并进行异构资源协同，而传统引擎在异构资源管理上相对薄弱。

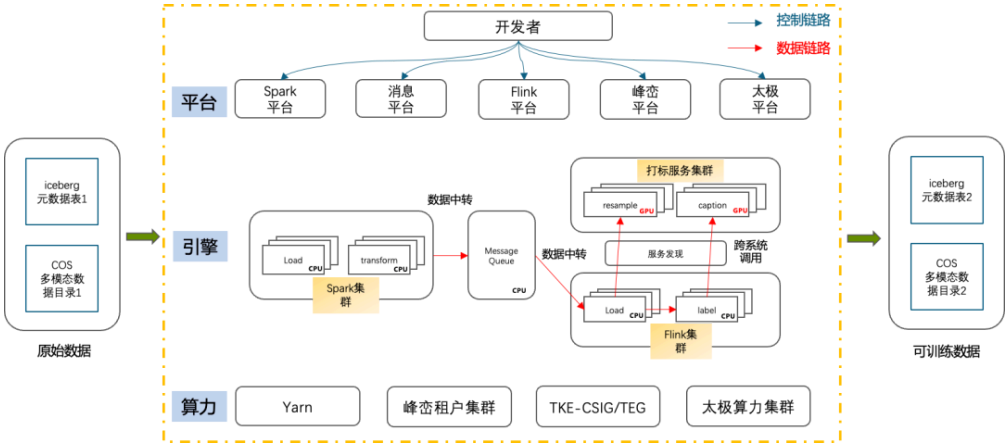
- **Python Native 生态需求：**AI社区的主流技术栈（PyTorch, HuggingFace等）均构建在Python生态之上。传统大数据系统依赖JVM（Java/Scala），虽然可以通过PySpark等方式调用，但存在跨语言序列化开销大、调试困难、依赖管理复杂等问题。构建Python Native的计算系统已成为AI基础设施演进的必然趋势。



pyspark架构，基于JVM生态，难以和AI生态无缝交互

3. 基于传统计算引擎的上一代数据管道

受限于传统计算引擎的计算范式和设计，我们上一代数据管道采用计算引擎与推理服务分离的架构，如下图：



上一代数据管道

整体上看，这是一个类lambda架构。首先通过一个spark任务读取hive/iceberg中的元数据，对数据进行转换后，写入MessageQueue，完成批转流；然后通过一个Flink任务消费上游数据，做管道业务逻辑的串联；在Flink任务的算子内部，通过北极星的方式对已经部署好的打标服务进行远程服务发现和调用，而真正的推理是跑在打标服务内部。

我们从三个维度分析该架构存在的问题：

从平台层来看，为了完成一个数据管道任务，业务方需要感知或创建多个平台的任务：通过us平台创建spark任务；通过消息平台申请和维护消息队列；通过flink平台创建和维护flink任务；通过峰峦分配的CPU资源创建CPU推理服务并在北极星维护；通过太极平台创建GPU推理服务。整个链路依赖的平台多，操作步骤复杂，难维护。

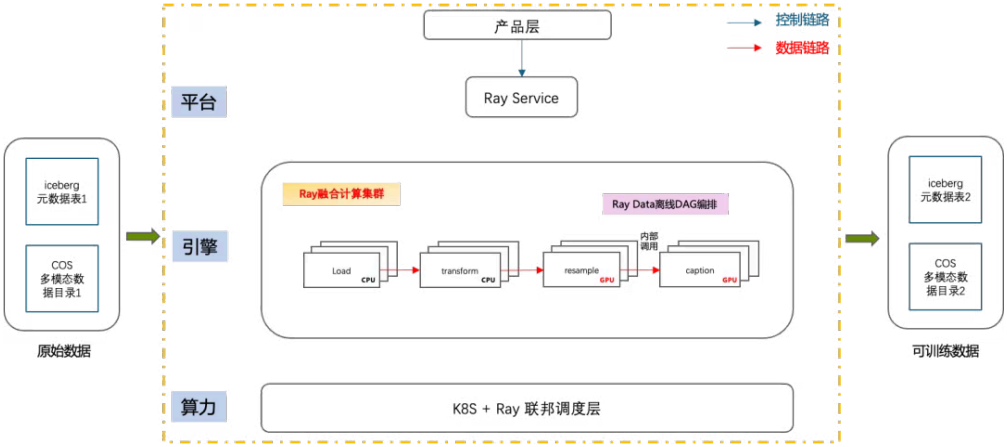
从引擎层来看，整个计算的运行时由三个计算系统批凑而成：Spark + Flink + 打标服务。三个计算系统存在系统孤岛问题，计算运行时协同困难，在解决扩缩容、流量反压、系统容错、版本升级与协议对齐等方面需要做大量的人工干预和跨系统优化工作，整体系统的运行效率上限并不高。同时数据需要在多个系统间进行中转，通信效率也受北极星等外部系统限制。

从算力层来看，不同的计算系统依赖不同的算力底座：如spark和Flink任务前期主要依靠yarn进行调度，CPU打标服务有些依赖峰峦租户集群，有些依赖TKE-CSIG/TEG集群，GPU打标依赖太极管理的算力集群。不同来源的资源难以共享，无法实现统一调度，无法实现运行时在算子之间动态调整资源。

二、基于Ray的新一代数据管道架构设计

1. 总体架构

考虑到上一代数据管道架构的局限性和问题，我们基于Ray构建了新一代数据管道。Ray作为一个专门面向AI场景的计算引擎，具备通用分布式、融合计算、异构资源调度、pythonic研发体验、AI生态化五大核心能力。其中Ray的融合计算能力可以完美解决上文提到的三个系统的孤岛问题，通过融合的方式将数据处理和推理的计算统一到一个系统和任务上运行：



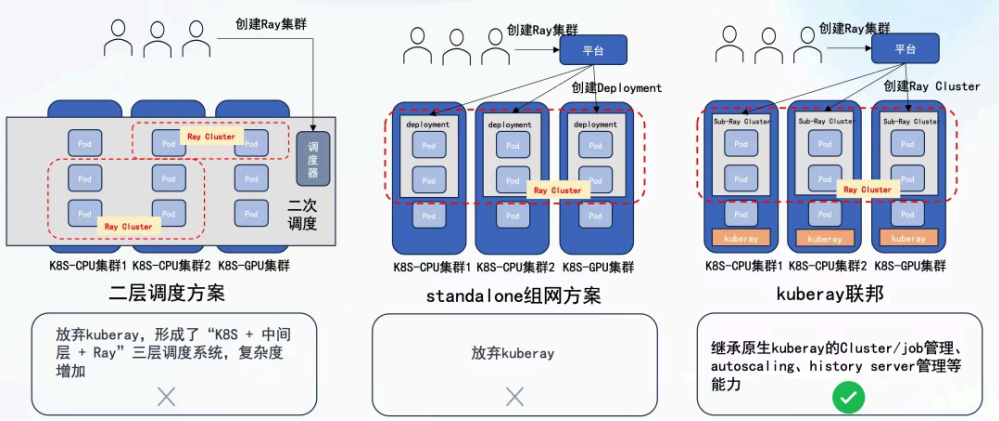
基于Ray的新一代数据管道

2. 核心架构解析

(1) 云原生调度融合：跨集群统一调度

在云原生环境下，我们面临物理集群碎片化的挑战：存在上百个Kubernetes物理集群，且CPU和GPU资源通常分布在不同的集群中。尽管Kubernetes联邦机制日趋成熟，但实现Ray Cluster跨物理集群边界的全局调度仍具挑战性。

鉴于此，实现新的管道架构的前提是需要具备跨多Kubernetes集群的单RayCluster调度能力。在技术选型阶段，我们对以下三种方案进行了深入评估：

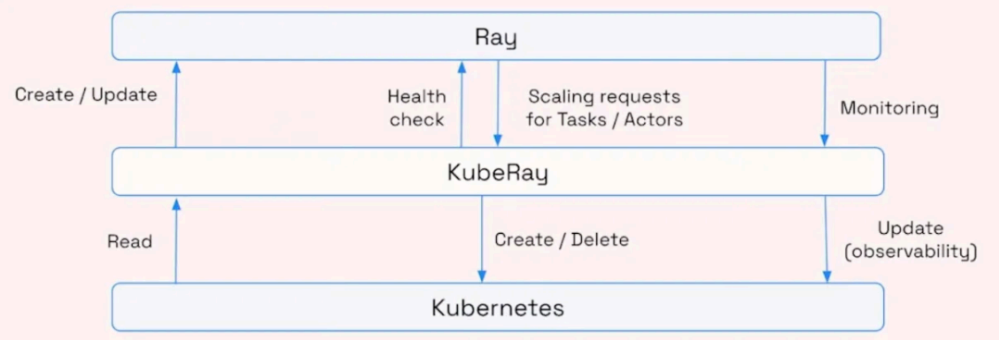


RayCluster跨多K8S集群调度方案选型

方案1 二层调度方案：做法是在各个Kubernetes集群申请大量Pod常驻，在这些Pod之上构建一层自定义调度器，用来调度Ray Cluster和Ray Job。这种方式引入了额外的调度层，系统从“Kubernetes+Ray”的两层调度变为“Kubernetes+中间层+Ray”的三层调度，增加了系统复杂度和调优难度。

方案2 平台层standalone组网方案：在平台侧直接调度多个Kubernetes集群的deployment形成多个Pod组； 在每个Pod内部启动Ray的进程， 通过指定其中一个节点为Head进行standalone方式的组网。这种方式虽然没有引入额外的调度层，但同样放弃了kuberay的能力，需要在平台层建设大量的工作替代kuberay的能力。

方案3 KubeRay联邦方案：保留原生KubeRay，通过扩展支持KubeRay集群联邦。

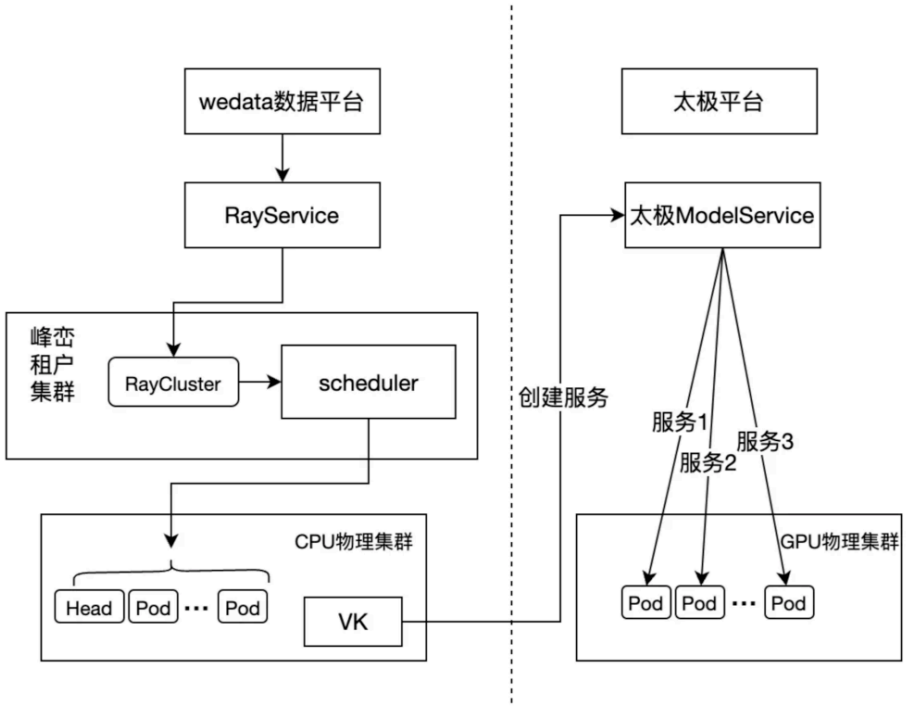


kuberay

Kuberay是Ray官方推荐的Ray operator实现，随着K8S+Ray的组合在业内的普及，Kuberay应用广泛，是Ray on K8S落地的不二之选。Kuberay不仅拥有Ray Cluster和Ray Job管理能力，还具备Ray的高可用和容灾能力，同时Kuberay也是Ray autoscaling链路上最成熟的实现方案。近期Ray社区还在推进history server的能力，也是强绑定在Kuberay上。我们的策略是开源路线，尽可能地兼容和复用社区已有能力，因此最终选择方案3，扩展KubeRay支持集群联邦。

KubeRay联邦架构的演进： KubeRay联邦架构演进经历了两个阶段：

- **阶段一：VK（Virtual Kubelet）架构**

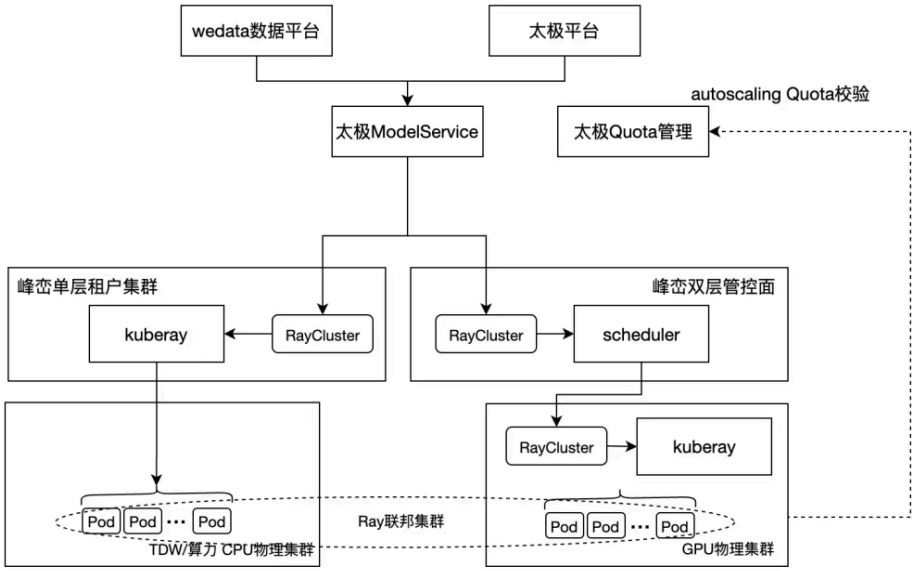


基于vk的架构

为了实现跨集群组网，我们实现了第一套基于Virtual Kubelet（vk）的架构。其实现方式是在CPU集群支持完整的Kuberay能力，然后通过vk将太极集群的GPU资源抽象成kubelet，创建GPU pod实际上调用的链路是通过model service创建太极服务，进而创建真正的pod。该方案在TEG内部落地，但在规模化中遇到比较明显的问题：通过vk创建太极服务，架构依赖颠倒（底层组件依赖上层服务），效率低，每次创建一个GPU pod都要创建一个太极服务，链路太重。在后期的业务落地过程中，Ray Cluster节点数量过百即会对太极链路造成较大压力，无法规模化生产。

- **阶段二：KubeRay联邦架构**

考虑到VK架构的瓶颈，我们推进了更合理的KubeRay联邦架构的落地。该架构在不同的Kubernetes物理集群并发部署KubeRay Workload，通过配置约束**仅单一Workload启动GCS（Global Control Store）**，其他workload的节点均作为Worker节点组加入集群。同时我们实现了联邦集群的autoscaling能力，通过联邦机制实现全局资源弹性。



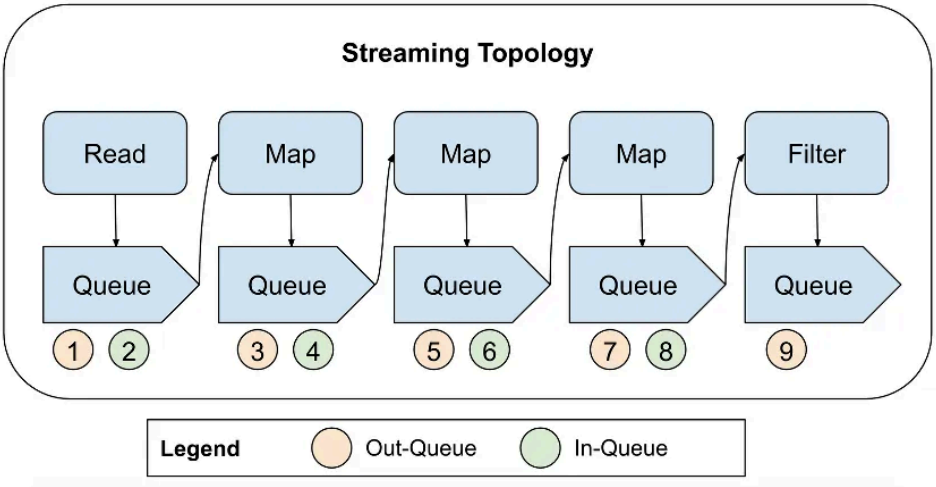
kuberay联邦架构

该架构改造完成之后，我们实现了异构资源调度的统一，即CPU算力和GPU算力可以通过一个入口实现灵活的Ray集群组网，同时wedata数据平台和太极AI平台可以实现复用相同的链路，**达到了Data + AI的调度统一。**

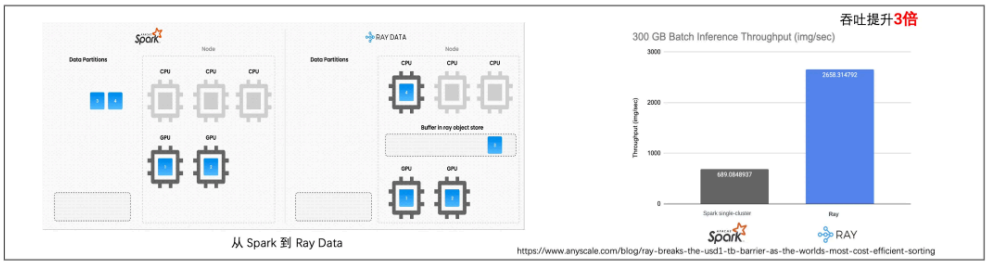
(2) 计算范式融合：Streaming-batch流水线

统一云原生调度后，接下来我们看在计算引擎维度，如何支撑数据管道的业务形态。上文提到，数据管道的本质是离线推理。而在我们上一代数据管道的架构中，是通过计算引擎访问在线服务的方式实现离线推理，这种方式通信效率低，无法实现反压机制，没有统一的中心调度器做全局资源调度。而Ray Data是专门面向离线推理（和训练）设计的计算引擎，近几年在字节、阿里、英伟达等公司已经有了大规模落地的案例。因此在新一代管道架构中，我们选择Ray Data开源路线。

为什么Ray Data适合做离线推理？这得益于其streaming-batch的计算范式。离线推理从大的需求上讲是批的需求，输入的数据往往是有边界的数据集。然而由于存在异构计算的需求，类似spark的BSP计算范式无法使其性能达到最优。当CPU计算时，为了最大化资源利用，GPU不能空闲，因此需要打破BSP的范式，实现流水线运行，这也就是streaming-batch范式的最大收益。在社区的异构计算测试结果中，Ray Data相比Spark可以拿到3倍的性能收益（Offline Batch Inference | Ray, Apache Spark & SageMaker）。

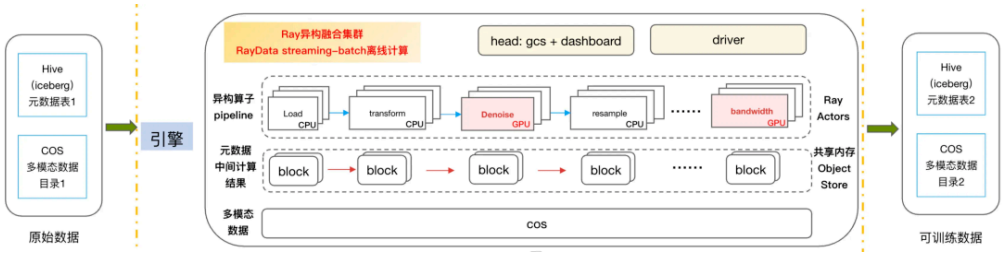


Ray Data的streaming-batch计算



离线推理Spark vs Ray Data（社区文章）

统一后的计算架构图： 从融合计算的角度来看，Ray Data融合了数据处理和模型推理两种计算，充分发挥了Ray的核心能力。融合后数据管道整体的架构如下：



RayData融合计算

其主要优势和特点：

● 所有的CPU计算和GPU计算、数据处理和模型推理都集成到一个统一的Ray Data离线计算任务中，不依赖外部的计算系统和服务系统

● 数据以dataset的形式在算子之间以streaming的方式流转，可以根据需求构建CPU和GPU算子的pipeline，依靠Ray的Actor和Task能力

● 算子之间以block的形式进行数据中转，依靠Ray强大的共享内存Object Store能力，可以实现大规模数据缓存在内存和磁盘；业务层依赖cos下载和上传多模态数据

● Ray通过全局的GCS进程进行集群管理和调度，通过全局的driver进程进行Ray Data 算子actor pool创建和data blocks分配，实现算子的弹性伸缩、反压以及细粒度的容错

三、Ray数据管道的关键技术优化实践

基于以上架构方面的建设，我们初步具备了接入混元数据管道的条件。但在面对大规模大语言模型和多模态模型数据管道的多样化场景时，各种各样的业务问题和系统难题催促我们不断优化。接下来从以下几方面介绍我们的优化工作：容错能力、利用率、规模化和可观测性。

1. 容错机制：构建高可靠数据底座

Ray数据管道整体是融合计算的思路，将复杂的数据处理和推理流程融合到一个Ray的任务里，因此Ray任务内部的处理逻辑同样具有一定的复杂性。在分布式系统中，随着节点规模扩张，故障常态化特征愈发显著，容错能力成为系统稳定性的基石。

(1) 增强基础容错能力

Ray社区版本已经具备了比较完备的容错机制。在业务接入实践中，我们对其做了进一步强化：

分类	社区版能力	内部版增强能力
Ray Core系统组件	Head节点高可用，worker节点故障替换 Task重试，Actor重调度 Object血缘回溯	Task/Actor调度超时机制 Object血缘回溯超时机制 节点故障动态感知
Ray Data框架层	依靠Ray Core层容错	Actor黑名单机制 算子执行超时机制

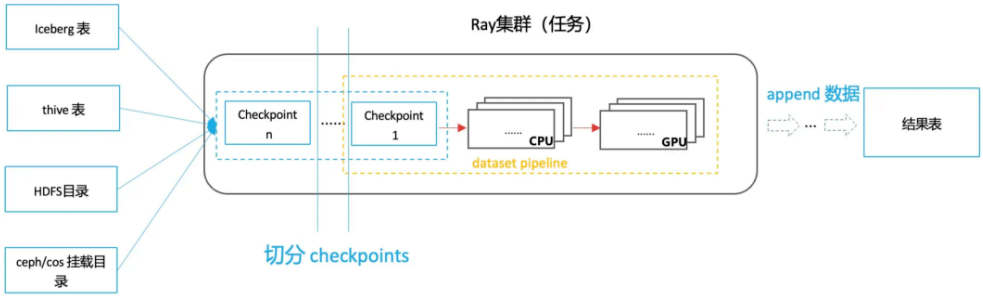
基础容错能力

首先在Ray Core层面，Ray可以实现除driver进程外任意一个进程或组件的快速恢复，同时也有比较完善的Task重试和Actor重调度能力。但在实际生产环境中，我们还是发现在一些极端case中，Task/Actor在调度中会永远pending，同时也会有部分节点持续无法成功调度新的进程。因此，我们对这部分能力进行了兜底：首先增加Task/Actor调度的超时机制，及时对pending的请求进行spill back，避免整个作业卡死；其次我们开发了插件化的节点故障检测框架，可以从多种维度（如磁盘、 网卡、 日志等）实时感知节点故障，并优化调度决策（屏蔽节点或触发Pod重调度）。另一方面，社区版本的Object血缘回溯机制较为复杂，在多算子场景经常出现回溯链路卡死的情况。因此我们也对Object血缘回溯链路增加了超时机制，快速丢弃长时间无法完成回溯的Object触发Ray Data作业层兜底。

在Ray Data框架层，社区版本主要依赖Ray Core层的容错能力。我们在内部版本Ray Data层也增加容错机制，从上层框架进一步保障系统的稳定性。首先我们建立了Actor的黑名单机制。因为Ray Data是把单个数据处理Task强绑死在一个Actor上，当Actor内执行Task失败时，就会引起部分数据计算失败。我们的做法是会把持续有Task失败的Actor加入黑名单，避免计算失败和数据丢失累加。其次在算子执行层面，我们常常发现用户的算子长时间无法返回（通常是算子实现问题），导致作业超时。因此我们也为算子执行增加了超时机制，方便用户做定制化决策。

(2) Checkpoint与断点续算

尽管Ray具备基础容错机制，但在极端故障引发Job级失败时，重算成本依然高昂。特别是在Iceberg数据湖场景中，为保障数据一致性，通常要求Dataset运行结束后再统一执行commit操作。若长周期任务在尾声阶段失败，将导致巨大的算力损耗。此外，部分业务场景存在提前抽样校验数据的需求。 为此，我们引入Checkpoint机制
，支持任务运行过程中的中间状态定期持久化入库，实现断点续算与中间结果可见性。



checkpoint实现

(3) 存算分离：

应对不稳定低优资源的架构重构

有了基础容错能力和checkpoint能力之后，我们已经可以支撑10w CPU和1k GPU规模的生产化任务。但随着业务的不断增长，离线推理任务必须要面临的问题是，承接公司内部各种不稳定的低优资源，优化资源使用率和利用率，扩大业务产能。不稳定低优资源包括：



腾讯内部不同类型的不稳定资源

- **弹性GPU卡：**在线业务quota内没有被分配出去的卡。特点是持有的时候比较稳定，但当在线业务抢占卡时，会出现批量回收，导致大批正在跑的节点和进程挂掉，同时作业可能长时间拿不到卡。

- **混部GPU卡**：与在线业务同机部署，依靠GPU混部技术实现高低优进程共享卡。特点是拿不到全量的GPU显存和算力，显存和算力的压制情况会随着在线Pod的情况波动，极端情况会出现推理进程卡死。

● **混部CPU：**在GPU资源上挖掘的CPU资源。特点上除了CPU会被压制外，为了不影响GPU的稳定性会限制Disk以及进程和线程（Pid）的使用量。

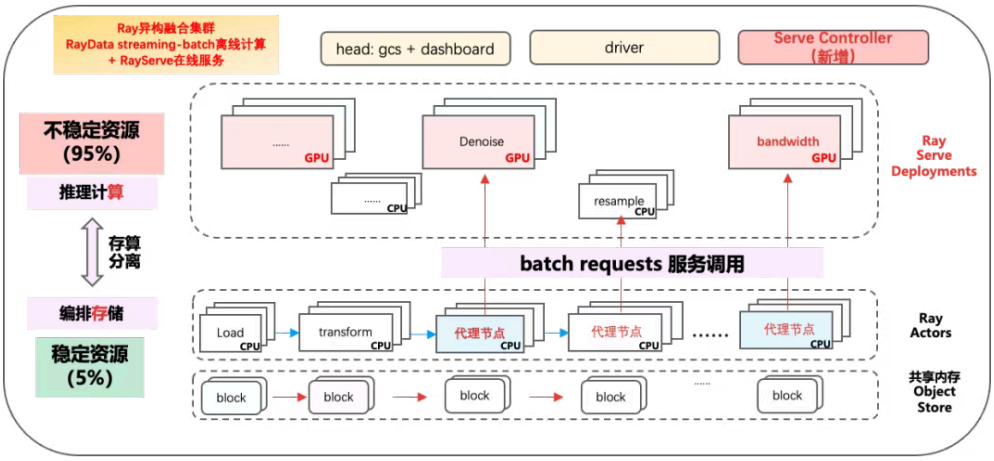
● **低优CPU：**与高优CPU共享的低优CPU服务。特点是CPU会被严重压制，基本上是质量最差的CPU资源，同时磁盘质量也比较差会导致io不稳定。

在默认Ray Data架构下，推理算子的中间数据以Blocks形式缓存在计算节点（Raylet）的Object Store（共享内存）中。此为典型的存算一体架构。当面临不稳定低优资源被回收或者节点卡死时，内存中的中间数据随之丢失，进而触发Ray的血缘重建（Lineage Reconstruction）机制。在深层计算图场景下（(如最长的pipeline会串联21个算子），血缘回溯成本极高，频繁的节点回收极易导致Raylet/Actor阻塞，甚至引发级联故障。

Dataset / Operator Name	Progress	State	Rows Outputted	Memory Usage (current / max)	Bytes Spilled	Logical CPU Cores (current / max)	Logical GPU Cores (current / max)
dataset_25	3016 / 10000	RUNNING	639042	13.56GB/15.22GB	0.0000B	5566.05/5676.4	721/721
Input0	1 / 1	RUNNING	0	0.0000B/0.0000B	0.0000B	0/0	0/0
Repartition1	10000 / 10000	RUNNING	0	0.0000B/0.0000B	0.0000B	0/0	0/0
MapBatches( )2	10000 / 10000	RUNNING	60000	760.18MB/3.74GB	0.0000B	0/0	0/0
MapBatches( )3	8907 / 10000	RUNNING	53184	1.44GB/2.28GB	0.0000B	107.80000000000001/271.40000000000003	0/0
MapBatches( )4	7600 / 10000	RUNNING	45342	1.08GB/1.36GB	0.0000B	230.75/230.75	0/0
MapBatches( )5	7230 / 10000	RUNNING	43230	1.99GB/1.99GB	0.0000B	0/0	360/360
MapBatches( )6	5558 / 10000	RUNNING	33156	1.65GB/1.66GB	0.0000B	480/480	0/0
MapBatches( )7	4565 / 10000	RUNNING	27270	677.43MB/706.47MB	0.0000B	0/0	41/41
MapBatches( )8	4550 / 10000	RUNNING	27138	568.42MB/664.04MB	0.0000B	0/0	64/64
MapBatches( )9	4481 / 10000	RUNNING	26748	308.34MB/317.56MB	0.0000B	0/0	32/32
MapBatches( )10	4467 / 10000	RUNNING	26634	282.32MB/330.96MB	0.0000B	0/0	32/32

多算子复杂任务示例

要彻底解决不稳定低优资源场景下的容错问题，降低容错成本，需要我们对计算架构进行又一次重构。因此，我们从存算一体的架构演进到了存算分离的架构。存算分离即将推理算子的计算进程（Actor）和计算结果（blocks）缓存进程（object store）分离，使其不绑定在同一个节点上。这样当计算节点异常或者回收时，已经计算好的blocks仍然在集群内保持可用，从而避免触发血缘重建。



Ray数据管道之存算分离架构

为了实现存算分离，我们为不稳定资源配置了占比5%（实际上占比可能小于5%， 目前20个推理实例仅需要配置1 core的稳定资源）的稳定资源，用来存储计算的中间数据。因为这5%的资源是纯CPU节点，其带来的资源成本是完全可以接受的。我们将dataset pipeline的所有stages的actor pool都部署在稳定节点，进而确保stages之间传输的blocks缓存在稳定节点的内存中。面向需要大量计算资源的CPU和GPU推理算子，我们将其以serve deployment的形式单独抽成services，部署在大量的不稳定资源上（如弹性GPU卡、 混部GPU卡、 混部CPU、 低优CPU等）。这样既完成了向存算分离架构的改造。其实现上有如下特点：

● **接口上对用户几乎无感知。**我们沿用了Ray Data已有范式，用户仅需要将map_batches调用改成map_batches_plus调用，然后通过环境变量切换存算一体和存算分离两种计算模式。用户代码不感知serve deployment，Ray Data参数会自动映射成deployment相关参数。

● 在存算分离架构上，在Ray Data调度器之外引入了**serve controller调度器**，负责所有serve deployment的部署、扩缩容和负载均衡。多一层调度器缓解了Ray Data调度器的压力，牺牲的方面是增加了一些调优的难度。

● 经过我们对新架构的打磨（如对serve controller进行调优、 定制化负载均衡算法等），新的架构在保证管线任务可以在不稳定资源上进行生产化之外，也实现了**性能零回退**（No Regression）。

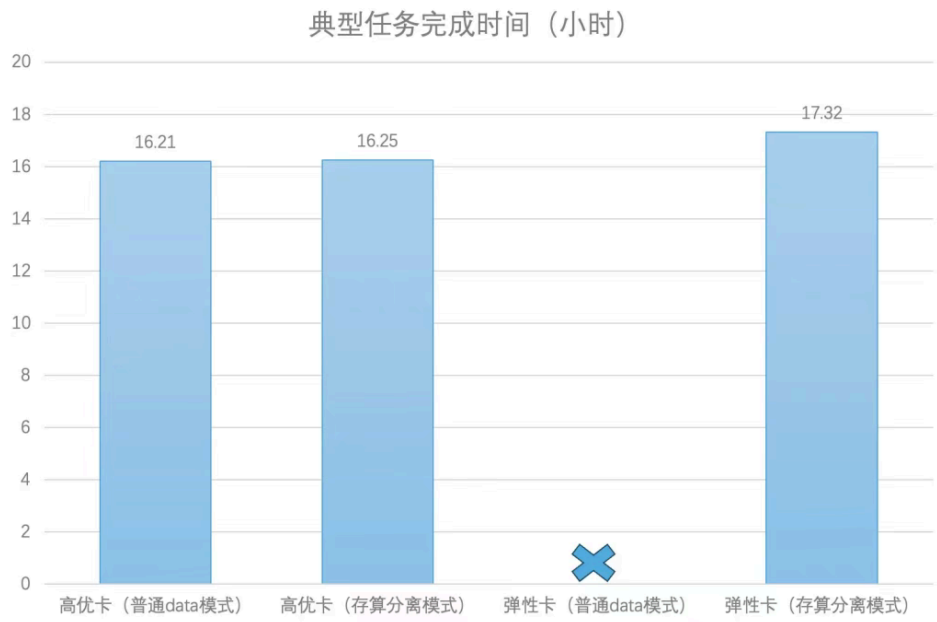


存算分离作业的GPU分配曲线（左）和 GPU利用率曲线（右）

存算分离架构的产能评估如下：

● 在高优卡场景，普通data模式的任务完成时间是16.21小时，存算分离模式的任务完成时间是16.25小时，性能接近，证明性能零回退（无损）；

● 在弹性卡场景，普通data模式由于频繁丢数据，无法完成任务，而存算分离模式下可以在17.32小时完成任务，性能较高优卡有损耗但可接受。



典型任务完成时间对比

2. 资源利用率：极致效能挖掘

资源利用率是反应数据管道产能的最重要指标之一，也是整个技术栈长期优化的主要目标之一。对于资源利用率的优化，计算引擎层面我们主要聚焦解决以下几个方向：



Ray内部版本资源利用率优化工作

● **冷启动优化：**数据管道是大规模的离线任务，任务启动中除了需要云原生调度层创建大量的Pod之外，在引擎启动后Ray Data任务达到最高throughput之前，需要经历数据加载、模型下载、进程池启动等多个阶段。对此我们做了如下优化：

○ **eager化setup环境：**支持在执行计划启动前进行环境预热，不同的运行时环境（如pip环境、模型文件等）在不同的节点组内提前创建；同时提前启动Actor pool的初始化，保证在数据流到下游之前具备一定的推理并发执行能力。

○ **合并序列化：**actor pool初始化过程中会对大批同构的actor做通信，这其中的序列化开销占用较大的CPU处理时间。我们通过对序列化数据进行合并，降低序列化计算量，将批量通信的时

间降低90%。

○ **指数扩缩容：**在社区版本默认的实现中，每一轮scaling up只能创建有限的actor（通常是1个），这在大规模算子启动场景跟不上数据流入速度。我们通过优化扩容策略，支持指数方式进行扩缩容，大大提升了算子冷启动的速度。

● **计算中间Bubble消除：**多算子计算场景和checkpoint计算场景下，往往会出现或多或少的计算中间Bubble问题。我们通过如下优化手段尽量消除Bubble：

○ **autoscaling优化：**Bubble的产生通常是因为随着数据的变化，算子actor pool的autoscaling没有及时跟进资源的调整。这里最核心的是Ray Data内部的actor pool利用率算法。社区原生的算法在计算利用率对pending actor的考虑欠佳，导致初始化逻辑较重的算子自动扩容较慢。我们对这部分算法进行了重构，支持更激进的策略对资源进行快速调整。在缩容方面，常见的问题是actor因为血缘问题或者GC慢问题导致长时间无法销毁，我们也针对该问题进行了特殊优化。

○ **Job之间常驻集群复用：**Job之间会因为资源的反复释放和创建产生资源利用率Bubble。在数据管道的场景上，离线任务通常是例行化的，一个任务一个任务串行执行，但实际上是使用相同一批资源。我们通过支持例行化任务运行在常驻集群上，可以大量缩减Pod的调度次数，以此快速利用上刚刚被释放的GPU。

○ **checkpoint之间进程池复用：**在同一个作业内部，我们通常使用checkpoint切分数据进行分批计算，因此checkpoint之间也会出现Bubble。为此，我们支持了在checkpoint间复用Deployment（Actor pool）的能力，大大降低了checkpoint之间的切换损耗。

● **长尾问题治理：**长尾问题在存在数据倾斜的任务上较为突出，同时在不稳定低优资源场景下，部分运行在不稳定资源上的算子也会形成长尾效应。为此，我们做了如下优化：

○ **算子实例timeout机制：**这是最直接的解决长尾问题的机制。用户可以根据自己的经验为不同的算子设置不同的timeout，通过提前中止算子实例的方式化解长尾问题。

○ **不健康算子实例自动检测：**除了手动设置timeout，我们还基于issue detector实现了自动检测。基于运行时统计数据（均值+标准差），自动检测并重启阻塞或异常缓慢Pending的算子实例。

○ **缩容效率优化：**缩容效率低也会引起长尾问题，我们在做了上文提到的缩容优化后，部分作业的长尾问题也得到了极大缓解。

● **算子计算效率提升：**数据管道中运行着大量的推理算子。在算子计算效率方面，我也做了不少工作对整体利用率进行优化：

○ **Triton Server集成：**数据管道存在大量的已经优化过的算子是集成到Triton Server中。Ray Data本身是和Triton Server解决的场景是不同的，一个面向离线，一个面向在线。 但为了复用已经优化好的算子，我们在Ray Data中封装了Triton Server算子的集成能力，可以快速将在线链路的算子“搬迁”到离线链路。

○ **复合算子拆分：**在实际的业务支持中，我们会发现部分算子是CPU + GPU的复合算子。部分复合算子是CPU bound类型的算子，比如数据管道的视频清晰度算子，在GPU机器上执行时，CPU上的切帧逻辑会把CPU打满，而GPU利用率只有20%。我们通过与业务团队一起对算子进

行改造，实现CPU预处理与GPU推理的彻底解耦，消除了CPU瓶颈对GPU计算的影响，将GPU利用率优化到70%以上。

○ **多算子资源碎片优化：**在多算子场景，通常不同的算子需要的资源量存在较大差异，比如有些算子需要1卡，有些算子需要4卡。这种情况如果不对调度进行优化，会造成4卡的算子无法利用全部的剩余资源。本质问题还是资源碎片问题。我们通过对不同类型的资源定制不同的调度策略（如CPU更适合平铺，GPU更适合堆叠），实现资源的最大化分配。

● **任务参数自动优化**：Ray Data管道任务中存在多个需要调整的参数，都会影响任务吞吐量和硬件资源利用率，对于初次接触平台的用户，学习门槛较高，为此，我们提供了如下工具：

○ **算子基线自动压测：**提供SDK给到业务同学，在算子注册到算子仓库的同时，平台会自动使用对应类型资源执行算子推理，通过资源利用率、显存使用等指标记录，即可得到最优吞吐量下的batch size大小，降低算子调试难度。

○ **任务基线自动测算：**基于DAG中多个算子的性能基线测算，按上下游 QPS 匹配计算各算子所需资源（以整数卡为粒度），通过加权合并多链路需求，得出整个任务的最小资源需求和整体QPS，使得多算子任务的调试优化任务变得简单。

○ **资源智能调度**：通过算子基线自动压测、任务基线自动测算等手段，获得任务的基线数据，再此基础上，业务提供期望完成时间，以及数量规模，即可评估出任务运行所需资源，进而在平台级别分配出对应的资源来保证任务运行，进一步提高任务上线效率。

3. 规模化能力：支撑千节点级生产环境

Ray社区一直致力于扩展Ray的规模化能力，目前官方benchmark的数据中， 单个Ray Cluster已经可以支撑数千甚至上万节点。 但经过沟通我们了解到，社区对规模化能力的benchmark还仅仅是证明组网能力，并没有对Ray Data这样的离线任务进行规模化评估。 Ray Data离线任务对系统的中心化调度能力有着极高的要求， 这里的影响因子比较复杂，最终的规模化能力不止跟节点数相关，还跟离线任务具体的实例规模、数据规模和算子复杂度息息相关。经过在内部的实践和优化，我们逐步沉淀出了Ray Data任务的准入标准，主要通过“Ray Cluster规模”、“Actor规模”、“blocks数量”、“算子复杂度” 等几个维度限制用户的使用范围：

安全基线	6月情况	8月底	当前
Ray Cluster	500节点	1k节点	2k节点
Actor规模	1w	2w	4w
blocks数量	2w	4w	12w
算子复杂度	5个算子	20个算子	20个算子

Ray Data任务准入标准

近半年，经过我们的优化，Ray Data的单集群规模可以从6月初的“500节点/1w Actors/2w blocks/5个算子”扩展到现在的“2千节点/4w Actors/12w blocks/20个算子”。规模化方面我的主

要工作如下：

● **系统组件负载优化：**在存算分离架构中，有GCS、Driver和Serve Controller三个中心调度组件。这三个组件的优化直接影响规模化的上限。

○ **Head节点与GCS：**通过参数化调优的方式优化GCS的CPU负载与内存占用，缓解元数据压力；为GCS高负载情况做兜底处理，比如对pub/sub数据丢失和fetch function的RPC超时等情况容忍，增强系统的鲁棒性。

○ **Driver进程：**重构和优化Driver端event loop内对actor和blocks的调度，防止大规模tasks和objects调度超负荷成为业务层调度瓶颈。

○ **Serve Controller：**对Controller进行参数化调优，我们发现高频的metrics会引起Controller内存持续增长，优化后效果明显； 通过自定义路由算法， 解决3000+ replicas场景下的负载不均衡问题。

● **弹性能力增强：**

○ **多级弹性：**通过联合wedata产品层、太极接入层、峰峦调度层，我们实现了多集群统一调度，同时也实现了异构集群的原生autoscaling能力。集群的autoscaling加上Ray Data调度层算子的autoscaling之后，形成了多级弹性的能力。我们还对弹性策略进行了增强，支持更平滑的缩容策略，避免在资源紧张的情况下GPU反复扩缩容被其他任务抢占。

○ **扩缩容性能优化：**通过支持指数级扩容等自研特性，优化超大规模集群中的扩缩容效率，提升整体资源利用率。

○ **集群热更新能力：**超大规模集群下的集群重启代价较大，目前我们正在逐步建设集群的热更新能力，当前已经支持修改节点数和autoscaling参数无需重启集群。

● **任务稳定性：**

○ **容错能力：**容错能力是超大规模集群任务稳定性的保障。如前文所述，我们在容错方面做了大量的工作，以此来保障在高频故障中超大集群可以快速恢复。

○ **OOM治理：**针对大规模数据表读取引发的OOM问题，我们实现Iceberg按Row Group拆分读取数据以及JSON文件按行切分Task，降低单Task的内存占用，配合Ray Data的反压能力，有效降低作业的OOM。

○ **进程cgroup隔离：**Ray本身是一种逻辑资源调度，actor进程之间默认没有隔离能力。在最新的社区版本中，已经支持通过cgroup v2对系统进程和用户进程进行隔离，以此保护系统组件的稳定性。但cgroup v2对内核版本有极高的要求，通常公司内的存量资源都无法满足。因此，我们基于社区的版本拓展了cgroup v1的支持，并推动在内部各云原生资源场景下的落地。

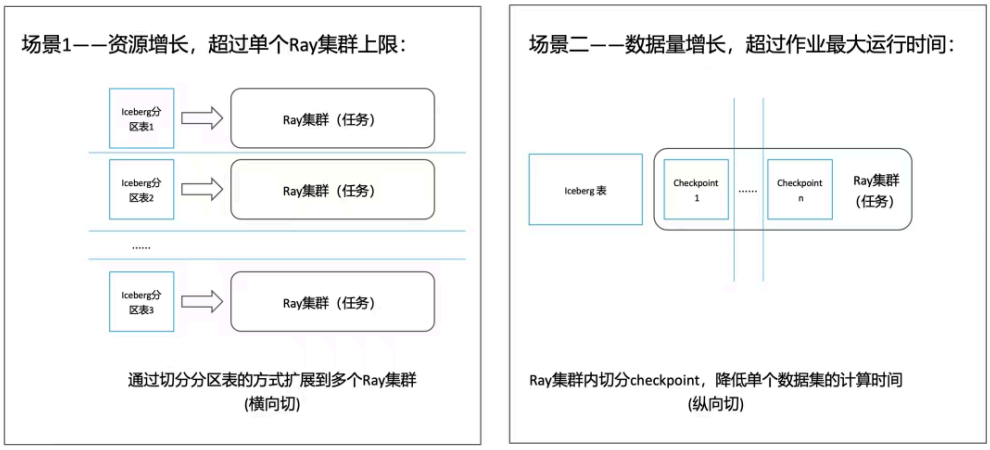
○ **进程数据目录：**在支持业务中我们发现，部分算子会在运行时使用大量的磁盘空间，而作业层无法保障对磁盘的及时清理。堆积在磁盘上的临时文件会破坏后续业务进程的稳定性。为此，我们支持了多维的进程数据目录。首先在峰峦层，会根据物理机情况分配一个最佳的数据目录给Pod，ray cluster 下线后自动清理；然后在Ray Core层，我们还会切分单独的目录给每个actor进

程，当actor进程退出后同样也会做自动回收。这样通过不同维度的目录管理最大化保障磁盘临时数据的及时GC。

在实际生产中，因AI趋势带来的业务增长是指数级的，在技术路线上，我们不会追求将一个数据管线的所有资源和数据永远放在单个Ray Cluster中处理。**当单个管线任务的规模超过我们的准入标准时，我们用以下两种方式做规模化扩展：**

- 场景1: 当管线的**资源增长**，超过单Ray Cluster上限时，我们通过将数据切分成多个分区表，每个分区表可以通过一个Ray集群做批量推理。这种方式可以称作**“横向切”；**

- 场景2: 当管线的**数据量增长，**超过作业最大运行时间时，我们通过在集群内切分checkpoint，降低单个dataset的计算时间。这种方式可以称为**“纵向切”。**

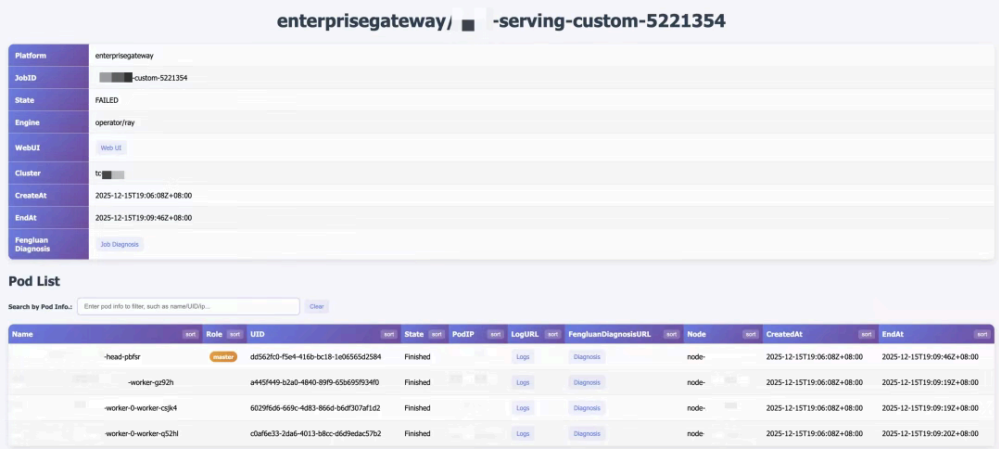


业务规模化扩展方式

4. 可观测性：提升研发运维效率

最后是可观测性方面。我们分别建设了以下能力：

- **峰峦Dashboard——云原生负载统一UI**

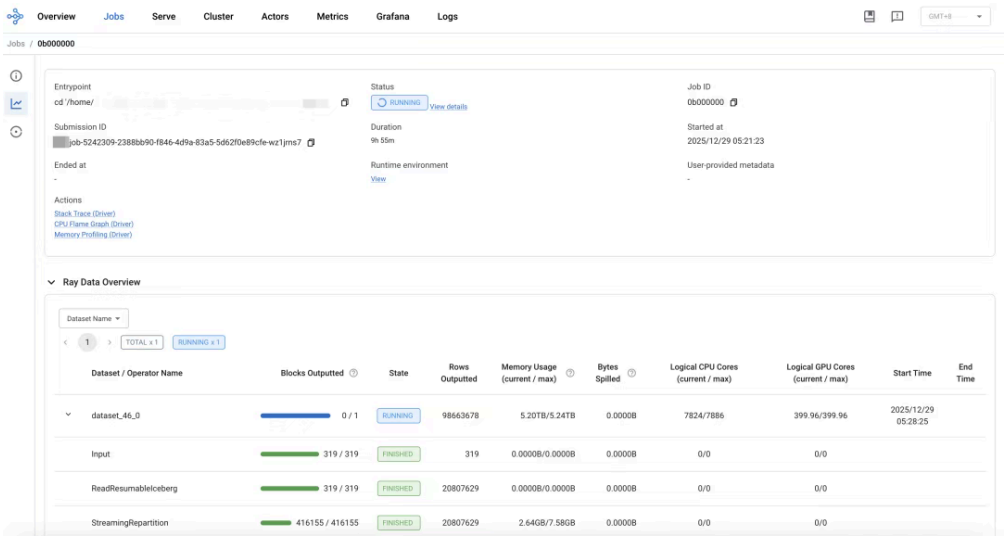


峰峦Dashboard

峰峦Dashboard是腾讯内部云原生架构峰峦的统一UI，支持多种计算引擎。峰峦Dashboard为用户提供了多层次的云原生元数据（如Pod列表、 Pod yaml等），方便用户快速了解workload状态和配置，同时支持持久化日志能力，在Pod删除和Ray集群删除情况下仍然可以继续提供查询服务。在没有Ray History Server之前，峰峦Dashboard就替代了一部分history server的能力。另外

在峰峦Dashboard上我们还集成了一键诊断能力，方便用户快速定位由于资源问题或者配置问题导致的Pod异常或者workload异常。

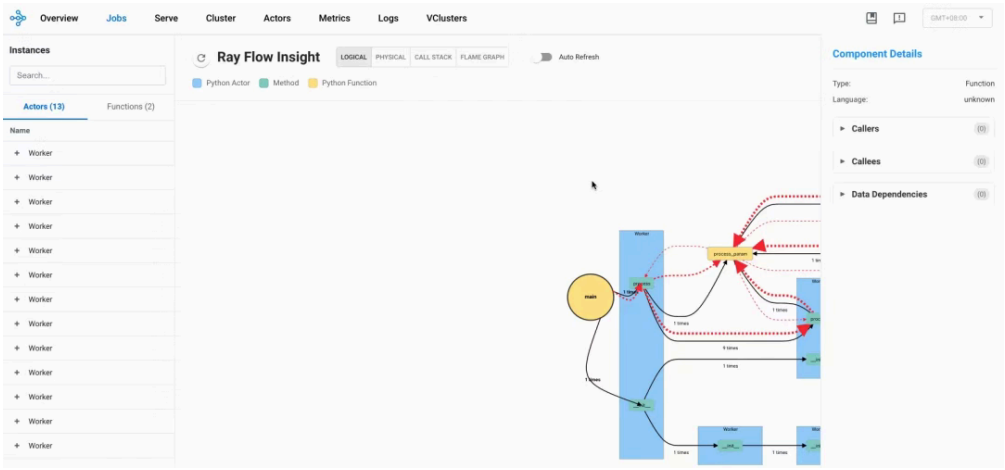
- **Ray Dashboard·——Ray统一UI**



RayDashboard

Ray Dashboard不用过多介绍，是Ray开源版本已有的统一UI。我们在内部也对其功能做了补充，比如支持完整的grafana页面、支持多磁盘、pid状态透出等。

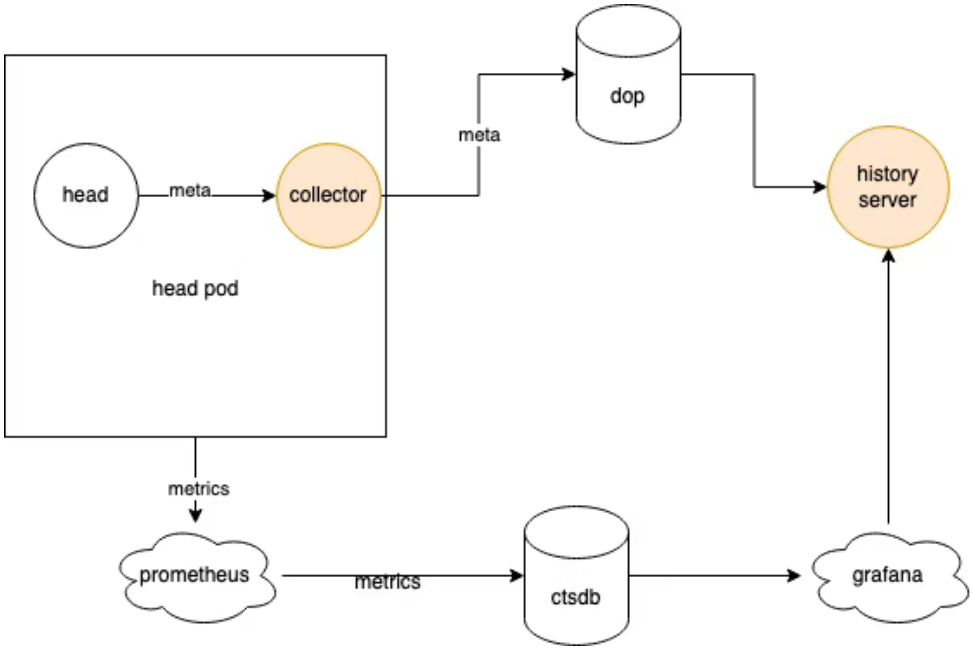
- **Ray Flow Insight——Ray多维运行时视图**



Ray Flow Insight页面

Ray Flow Insight是蚂蚁开源的Ray多维运行时视图，主要从Ray Core的角度进行设计，所以具备很强的通用性。Ray Flow Insight主要有几种视图能力：逻辑视图、物理视图、分布式火焰图、分布式调用栈等。在Ray社区版本中，目前还没有集成Ray Flow Insight。我们已经从Ant-Ray开源仓库中将该框架的实现引入到内部Ray版本。

- **Ray History Server——彻底解决Ray可观测性问题**



History Server架构

Ray History Server已经是社区讨论多年的问题，也是Ray的开发者呼吁最强烈的需求。目前该项目由阿里主导，正在社区做prototype。我们联合阿里云团队，第一时间将还在开发中的History Server在腾讯内部落地。

四、数据管道重构收益

经过以上我们在Ray异构融合底座方面的建设与实践，Ray已经成为混元数据管道的统一底座。半年内我们已经完成了10+条数据管线的重构，实现万卡规模的生产化。经评估，Ray可以帮助音频管线节省90%的GPU卡，为多模态理解管线
提升GPU利用率3倍以上，为文本管线提升40%的推理成功率，为数据管道整体提升研发效率1-3倍。

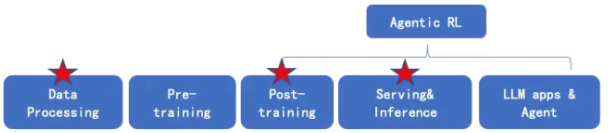


重构收益

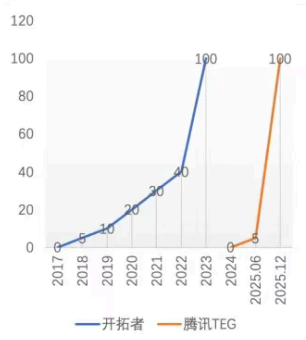
五、关于我们

我们是腾讯TEG-Ray团队，正在招聘Ray分布式计算工程师，感兴趣的朋友请通过graysong@tencent.com联系。我们的优势是会结合“GPU算力+K8S+Ray+训练/推理框架”多层能力进行co-design，因此像Ray集群联邦架构这种大规模的重构工作在我们团队内可以形成闭

环，快速落地。后续我们会持续在多模态数据处理、大模型在线推理、AgenticRL超融合架构等方向持续探索，同时会在今年开始加强与开源社区的共建工作，欢迎志同道合的朋友加入。



项目	资源融合	计算范式融合	框架融合
多模态数据管道	CPU+GPU	数据处理+推理	Pyiceberg+Ray Data
强化训练	CPU+GPU	推理+训练	vLLM/SGLang + FSDP/Megatron-LM
在线推理	CPU+GPU	推理服务P+推理服务D	vLLM



100w核 业务发展曲线





阅读 4346

留言

写留言

还没有留言，来写首评